

RETAINIFY — Employee Attrition Prediction System

Project Statement

The objective of this project is to analyze workforce data to uncover employee attrition patterns, retention risk factors, and organizational trends using machine learning and explainable AI techniques. The goal is to help HR teams and business leaders identify at-risk employees early, understand the drivers behind turnover, and take data-driven action to improve workforce retention.

Expected Outcomes

- Understand and preprocess IBM HR Analytics data for attrition modeling
 - Explore patterns in employee satisfaction, compensation, tenure, and role-level attrition rates
 - Train and evaluate multiple ML models optimized for high recall at a custom 0.3 threshold
 - Generate explainable predictions using SHAP values for individual employees
 - Provide retention recommendations and what-if simulation through an interactive web platform
 - Summarize findings through a deployed full-stack enterprise dashboard
-

Dataset

Source: IBM HR Analytics Employee Attrition Dataset. Contains 1,470 employee records covering 35 features including demographics, job role, compensation, satisfaction scores, and attrition status.

Modules Implemented

1. Data Acquisition and Understanding
 2. Data Cleaning and Preprocessing
 3. Feature Engineering and Scaling
 4. Model Training and Hyperparameter Tuning
 5. Model Evaluation and Threshold Optimization
 6. Attrition Driver Analysis
 7. Employee Trends Visualization
 8. Deployment Artifacts and API Development
 9. Frontend Dashboard and Interactive Platform
 10. Documentation and Presentation
-

Implementation Plan

Phase 1: Data Foundation and Cleaning

- 1: Data Acquisition and Understanding

- Define project goals, KPIs, and modeling workflow
- Load IBM HR CSV dataset using pandas
- Explore schema, data types, feature count, and null values
- Confirm 1,470 records across 35 columns with zero missing values
- Document the 84% retained / 16% attrited class imbalance

2: Data Cleaning and Preprocessing

- Drop constant-value columns: EmployeeCount, StandardHours, Over18, EmployeeNumber
- Encode the target variable Attrition as binary (Yes → 1, No → 0)
- Confirm cleaned dataset shape: 1,470 rows × 31 columns
- Save preprocessed output as processed-data.csv

Deliverables:

- processed-data.csv
 - Null audit report and schema documentation
-

Phase 2: Feature Engineering and Model Preparation

3: Feature Engineering and Scaling

- Engineer 6 derived features: IncomePerYear, PromotionGap, CompanyExperienceRatio, ManagerStability, WorkloadScore, SalaryGrowth
- One-hot encode 7 categorical columns: BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, OverTime
- Apply StandardScaler fitted on the engineered feature matrix
- Save feature_columns.pkl and scaler.pkl for backend inference
- Confirm final feature count: 50 columns

4: Class Imbalance Handling

- Apply SMOTE to the training split to rebalance 84/16 class distribution to 50/50
- Validate resampled class counts before model training

Deliverables:

- feature-engineered-data.csv
 - feature_columns.pkl
 - scaler.pkl
-

Phase 3: Model Training and Evaluation

5: Model Training and Hyperparameter Tuning

- Train three models: XGBoost, Random Forest, Logistic Regression

- Apply GridSearchCV with StratifiedKFold (5 splits) optimizing Recall
- Record best hyperparameters for each model
- Retrain final models on the full dataset using best parameters

6: Model Evaluation and Threshold Optimization

- Evaluate all models at custom classification threshold of 0.3
- Record Accuracy, Recall, Precision, F1 Score, and AUC-ROC per model
- Generate confusion matrices for each model
- Plot ROC curves, Recall comparison chart, and Accuracy vs Recall trade-off
- Save production models: xgboost.pkl, random-forest.pkl, logistic-regression.pkl
- Save shap_explainer.pkl using TreeExplainer on the final XGBoost model

Deliverables:

- xgboost.pkl, random-forest.pkl, logistic-regression.pkl
- shap_explainer.pkl
- Evaluation charts and performance summary table

Phase 4: Attrition Analysis and Visualization

7: Attrition Driver Analysis

- Analyze feature importance across XGBoost and Random Forest
- Compute Logistic Regression coefficients for explainability
- Calculate attrition rates by department, job role, overtime status, age bracket, marital status, and income band
- Identify critical system signals: overtime concentration in high-risk segment, income gap between risk tiers
- Compute workforce health index and financial liability estimates
- Export attrition_drivers_analysis.csv and model_benchmarks.csv

8: Employee Trends Visualization

- Plot attrition distribution, overtime impact, department breakdown, and role vulnerability index
- Generate age vs attrition histogram, income distribution by attrition, satisfaction trends
- Visualize tenure, promotion gap, stock option level, and job level attrition rates
- Produce correlation heatmap and top 15 features correlated with attrition

Deliverables:

- attrition_drivers_analysis.csv
- model_benchmarks.csv
- Visualization charts across all key workforce dimensions

Phase 5: Backend API Development

9: FastAPI Backend

- Build REST API with endpoints: /upload, /predict, /simulate, /batch-analysis, /performance, /feature-importance, /dataset-analytics
- Implement model loader with startup auto-loading of all six pkl artifacts
- Build preprocessing pipeline matching the exact feature engineering from training
- Integrate SHAP explainer to return top 6 prediction factors per employee
- Generate dynamic retention recommendations based on employee input values
- Auto-load IBM HR dataset from datasets/ folder on startup for instant analytics
- Configure CORS for cross-origin frontend access

Deliverables:

- Fully functional FastAPI backend
- requirements.txt
- All six pkl files in trained_models/

Phase 6: Frontend Dashboard

10: React Frontend

- Build five-page application: Home, Predict, Dataset, Model Performance, Batch Analysis
- Implement global state management using React Context API
- Upload overlay with drag-and-drop CSV/XLSX support
- Prediction form with model selector, personal profile, job details, and satisfaction sliders
- Post-prediction two-column layout: Risk Assessment and Explanation left, Retention Simulator right
- Live Retention Simulator calling /simulate on every slider change
- Dataset page with automatic EDA charts, imbalance analysis, and dataset profile table
- Model Performance page with threshold strategy, ROC curves, recall comparison, and model guidance matrix
- Batch Analysis page with workforce health index, financial liability, department heatmap, employee results table, and action plan

Deliverables:

- Fully deployed React application on Vercel
- All five pages functional and synchronized via shared state

Phase 7: Deployment and Documentation

11: Deployment

- Deploy FastAPI backend as a Web Service on Render
- Deploy React frontend on Vercel with REACT_APP_API_URL environment variable
- Push all pkl files, dataset, and source code to GitHub repository
- Validate end-to-end prediction flow on production URLs

12: Documentation and Presentation

- Write project README with setup instructions, architecture overview, and deployment guide
- Create UML class diagram, component diagram, and project workflow diagram
- Document all API endpoints with request/response schemas
- Record insights, model performance summary, and business impact findings

Deliverables:

- Live production URLs (Vercel + Render)
- GitHub repository with full source code
- README, architecture diagrams, and final presentation

Tech Stack

Data Handling: pandas, numpy, scikit-learn, imbalanced-learn

Machine Learning: XGBoost, Random Forest, Logistic Regression, SHAP

Backend: Python, FastAPI, Uvicorn, Joblib

Frontend: React, TypeScript, Tailwind CSS, Framer Motion, Recharts, Axios

Deployment: Render (backend), Vercel (frontend), GitHub (version control)

Documentation: Jupyter Notebook, GitHub README